2026

Programming 3B (PROG7312) - POE

THABANG THAMAE
ST10442590

Table of Contents

1. Project Completion Report *1*

1.1 What the final submission set out to do **1**

1.2 Challenges encountered and how they were overcome **1**

1.3 What I learned **3**

2. Technology Recommendations *5*

3. Rationale *6*

Repository *8*

References *9*

1. Project Completion Report

1.1 What the final submission set out to do

Parts 1 and 2 of this portfolio built a telemetry gateway that could ingest readings, score them for anomalies, and process operator commands in real time. Both were blind to one thing: the network itself. The dashboard could tell an operator that node ESP32-X23 had stopped reporting, but not whether it had failed or whether the relay carrying its traffic had failed, and not what else would go dark if that relay stayed down.

The final submission closes that gap with four structures: an AVL tree for per-device history, a binary min-heap, an adjacency-list weighted graph with breadth-first and depth-first traversals, and Prim's algorithm for the minimum spanning tree. The deliverable is not the structures in isolation but the question they jointly answer is if this relay dies, who re-routes, by what path, and what does the backbone then cost?

1.2 Challenges encountered and how they were overcome

Telemetry arrives in the worst possible order for a binary search tree.

The first design decision was also the first problem. A binary search tree keyed on timestamp is the natural structure for time-range history, but telemetry keys arrive in strictly ascending order. Every insertion is larger than everything already stored, so every insertion descends the right spine and the tree degenerates into a linked list. The O(log n) search the structure was chosen for becomes O(n).

This is the textbook pathological case, and it is exactly the case this data produces. The resolution was an AVL tree, where each insertion walks back up its own insertion path, recomputes subtree heights, and rotates wherever a balance factor reaches ±2. What I had not anticipated was how often that fires: because the input is monotonic, nearly every insertion triggers a rotation, and the deployed system reports 806 rotations across 946 insertions. My instinct was that a rotation rate approaching 1:1 indicated a bug. It does not, it is the structure doing continuously and cheaply what it was selected to do, and the cost of each rotation is constant while the cost of the degradation it prevents is linear.

To make the argument checkable rather than asserted, the tree computes and reports unbalancedHeight, the height a plain BST would have reached on the same input, alongside its real height. At 399 readings those figures are 399 and 9. Exposing the counter was more persuasive than any amount of prose about complexity classes.

Prim's algorithm needs an operation the .NET priority queue does not have.

Part 2 used PriorityQueue<TElement, TPriority> for the alert queue and it was the correct choice there. Prim's algorithm broke it. As the spanning tree grows, the

cheapest known cost of attaching a vertex still outside the tree can drop, so the frontier needs decrease-key: lower an item's priority in place. PriorityQueue<,> offers no such operation.

The common workaround is to push the vertex again at its new priority and discard stale entries when they surface. That works but inflates the heap from at most V entries to as many as E. I wrote MinHeap<TItem, TPriority> instead, maintaining a Dictionary<TItem, int> that maps each item to its current index in the heap array. A push for an item already present becomes a constant-time index lookup followed by a logarithmic sift-up. The subtlety that cost me the most time was that every swap during sift-up and sift-down must update both dictionary entries; miss one and the index silently goes stale, after which decrease-key quietly operates on the wrong element.

An algorithm that produced a plausible wrong answer.

The most instructive failure in this submission was a bug in Prim's that did not announce itself. My first implementation looked up the current best-known cost of reaching a neighbour through a helper that returned positive infinity unconditionally. Every relaxation therefore looked like an improvement. The algorithm still terminated, still visited every vertex, still produced a connected acyclic subgraph, and still reported a total weight that looked entirely reasonable. It simply was not the minimum spanning tree.

Nothing about the output flagged it. There was no exception, no empty result, no obviously silly number. What caught it was deciding in advance what the result had to satisfy structurally: a spanning tree over n vertices has exactly n − 1 edges, and every vertex must be reached. The endpoint now reports vertices, edges and isComplete, and 25 vertices against 24 edges with isComplete: true is the check. The fix itself was to track the best-known weight and its source vertex in two dictionaries maintained outside the heap, because a heap can be asked for its minimum and nothing else, there is no way to interrogate it for the priority it currently holds for an arbitrary item.

The transferable lesson is that for an algorithm whose output is hard to eyeball, correctness must be checked against an invariant rather than against plausibility. I would not have found this by looking at the number.

Evidence that proved nothing.

The AVL range query is justified by pruning: at any node whose key is already below the lower bound, the entire left subtree can be skipped. My first attempt to demonstrate this reported 62 comparisons on a tree of 62 nodes returning 62 matches, numerically correct and completely useless as evidence, because the query window happened to span the entire tree. With nothing to prune, a correct implementation visits every node.

Two changes fixed it. First, CompareRangeStrategies() now runs the same window twice, once through the tree and once as a linear scan and returns both comparison counts from the same request so the two cannot be compared across different data. Second, the demonstration uses a deliberately narrow window, ten seconds out of roughly five minutes of history, which produces 20 comparisons against 399 for identical results, about 95% pruned. The measurement was always valid; it was the experiment that was badly designed.

A failure simulation that reported nothing had failed.

Taking a relay offline and re-running the traversals initially reported zero re-routed devices, on a failure that visibly changed the mesh diagram. The cause is a property of mesh networks specifically: because redundant links are the entire point of a mesh, a node that loses its uplink usually has an equal-length alternative. It is still three hops from the gateway, but via a completely different relay. Comparing hop counts alone cannot see that.

The comparison now tests the next hop as well as the hop count, and a change in either marks the node as re-routed. Failing ESP32-S18 then correctly reports four nodes switching their uplink to ESP32-V21, the backbone cost rising from 186.4 to 194.6, and the mesh healed in 0.214 ms. The bug was not in any data structure, it was in my model of what "re-routed" means and only running it against a realistically redundant topology exposed it.

Framework and tooling friction.

Two problems cost time without being conceptually interesting, and both are worth recording because they recur. Razor reserves <text> as a control element, so emitting SVG text nodes with attributes fails to compile with RZ1023; the mesh diagram emits them through (MarkupString) instead. And the build fails with MSB3027 whenever the API is still running, because the process holds a lock on its own executable, the API must be stopped before every rebuild.

More seriously, a file silently failed to save on two separate occasions. The first time, in Part 1, Devices.razor was never written to disk and the submission shipped without its registry page; I only discovered it while working on Part 2. The second time, NavMenu.razor refused repeated writes with no error of any kind, leaving the Mesh Topology navigation link inactive even though the page itself worked. I now verify every file write by reading the file back before rebuilding, rather than trusting that a save succeeded. That habit is cheap, and it would have prevented a lost mark in Part 1.

1.3 What I learned

Choosing a data structure is a claim about the input distribution, not a preference. An AVL tree is not categorically better than a BST, and a Dictionary is not categorically better than a list. The AVL tree earns its place here specifically because telemetry

timestamps arrive monotonically, which is the one input order under which an unbalanced BST is at its absolute worst. Had readings arrived in random order, a plain BST would have been adequate and the rotation machinery pure overhead. The justification lives in the data, not in the structure.

Adjacency lists and matrices are a density decision. A 25×25 matrix is 625 cells holding 71 links. Mesh networks are sparse by physical necessity, a radio has a handful of neighbours, not hundreds, so E grows roughly linearly with V rather than quadratically, and the list's O(V + E) footprint tracks reality while the matrix's $O(V^2)$ does not. At the thousands of nodes, the brief describes, that is the difference between a few thousand entries and a million mostly-empty cells.

Queues and stacks are not interchangeable even when both traverse everything. BFS and DFS differ only in which collection holds pending work, and both cost O(V + E), yet they answer different questions. Because BFS reaches every vertex by the fewest possible hops the first time it reaches it, recording each vertex's discovering parent yields shortest paths for free, which is the entire basis of the routing feature. DFS, plunging down one branch at a time, answers connectivity and reachability instead. I also implemented DFS with an explicit stack rather than recursion, because recursion depth on a graph is bounded by the vertex count and a large mesh would risk a stack overflow on a path an explicit stack handles in heap memory.

Separating topology from liveness makes simulation non-destructive. Keeping an _online set alongside the adjacency list, rather than deleting failed vertices, means a failure can be injected, measured, and undone with the graph unchanged. Deleting and rebuilding would have been simpler to write and would have made repeated what-if analysis impossible.

Instrumentation is part of the deliverable. Every structure in this submission reports its own internal state, tree height, rotation count, comparisons per operation, MST completeness, re-route time. That was originally for the marker's benefit, but it is what found two of the bugs described above. A structure that cannot report what it just did cannot be shown to be working, and in the Prim's case could not be shown to be broken either.

2. Technology Recommendations

These are the changes I would make to take this system from a working demonstration to something that could be deployed against real hardware. The rationale for each follows in Section 3.

	Recommendation
1	Replace the synthetic link weights with RSSI-derived link quality
2	Replace MQTT-less HTTP ingestion with an MQTT broker
3	Move telemetry history from in-memory AVL trees to a time-series database
4	Replace client polling with SignalR server push
5	Maintain the MST incrementally rather than recomputing it
6	Add distributed state and horizontal scale-out via Redis
7	Instrument with OpenTelemetry rather than bespoke counters

3. Rationale

1. RSSI-derived link weights. The current topology is explicitly synthetic: LinkCost() derives each weight deterministically from the two device identifiers, so the mesh rebuilds identically on every run, which makes the measured figures in this report reproducible. Real ESP32 radios report RSSI, packet loss and retransmission counts, and a weight combining those would make the MST reflect actual transmission cost rather than a stand-in. Critically, none of the graph code changes, the algorithms consume a weight and are indifferent to where it came from. That separation was deliberate and is what makes this recommendation a configuration change rather than a rewrite.

2. MQTT for device ingestion. HTTP POST per reading is the wrong transport for battery-powered sensors. Each request carries full header overhead and requires a connection handshake, and the devices in question are power constrained by definition. MQTT's persistent connection and publish-subscribe model is the industry standard for exactly this reason, and its quality-of-service levels handle the intermittent connectivity a mesh experiences. The existing ingestion endpoint would become an MQTT subscriber; the archive, index and history services behind it are unaffected.

3. A time-series database for history. The AVL trees are the right structure for the problem the brief sets, and they demonstrate the balancing and range-query behaviour the module is assessing. They are not a durable store: everything is lost when the API restarts, and memory grows without bound as history accumulates. A purpose-built time-series store such as TimescaleDB or InfluxDB provides persistence, automatic downsampling of old data, and retention policies, and internally uses the same ordered-index principles the AVL tree demonstrates, which is why understanding the structure matters even when delegating it. The in-memory trees would remain as a hot cache for recent readings.

4. SignalR instead of polling. Every dashboard page currently polls the API on a timer. With one operator that is harmless; with fifty, the gateway serves fifty redundant requests per second for data that may not have changed. SignalR pushes updates only when something changes, which also removes the inherent latency floor of a polling interval, an operator currently learns about a critical alert up to one poll period after it fires.

5. Incremental MST maintenance. Prim's algorithm recomputes the entire spanning tree from scratch, which at 25 vertices completes in well under a millisecond and is entirely acceptable. At thousands of nodes with links changing continuously as radio conditions shift, full recomputation on every change is wasteful, because a single link change usually perturbs only a small region of the tree. Dynamic MST algorithms maintain the tree through edge insertions and deletions in polylogarithmic time. This is the one recommendation I would defer longest: full recomputation is simpler,

provably correct, and fast enough until the node count grows by two orders of magnitude.

6. Redis for distributed state. Every service in the gateway is a singleton holding in-memory state, which means exactly one API instance can run. That is a single point of failure in front of infrastructure whose whole purpose is redundancy, the irony of a mesh-monitoring system with no redundancy of its own is not lost on me. Moving the command stream, the suggestion model and the device index into Redis would allow several gateway instances behind a load balancer to share state.

7. OpenTelemetry instrumentation. The counters this system exposes, rotations, comparisons, re-route timings, were written by hand for this assignment and they served it well. In production they would need to feed dashboards, alerting and distributed traces, and reinventing that plumbing per metric does not scale. OpenTelemetry is the vendor-neutral standard and is natively supported in .NET, so the existing counters would be re-expressed as instruments rather than replaced.

Repository

https://github.com/ST10442590/prog7312_smartx_poe

References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. Introduction to algorithms. 4th ed. Cambridge, MA: MIT Press.

Microsoft, 2024. PriorityQueue<TElement,TPriority> class. [online] Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.priorityqueue-2 [Accessed 13 November 2026].

Sedgewick, R. and Wayne, K., 2011. Algorithms. 4th ed. Upper Saddle River, NJ: Addison-Wesley.